

Tests for variable importance estimation

Stacy Cyrille

2026-05-27

Table of contents

```
library(tidyverse)
library(MASS)
library(knitr)
library(survival)
library(timereg)
library(survML)
library(survivalSL)
library(SuperLearner)
library(randomForestSRC)
library(survex)

source(here::here("R", "utils.R"))
source(here::here("R", "generate_data.R"))
source(here::here("R", "pred_generators.R"))

# Set seed for reproducibility
set.seed(2026)
# n <- 200000
n <- 500
V <- 5
sample_split <- TRUE
vims <- c("AUC", "brier")
```

1 Scenario 1: Additive hazards

1.1 Objectives

This scenario:

- evaluates the overall performance of the VIM procedure under additive hazards: bias, confidence interval coverage, confidence interval width, rejection probability
- assesses the robustness of the VIM procedure to nuisance-model choice required for VIM computation: Aalen model (`timereg::aalen`), survival Super Learner (`survivalSL::survivalSL`), global survival stacking (`survML::stackG`)

1.2 Data generating mechanism

Four binary covariates were generated *independently* with prevalences spanning rare to frequent alterations, $X_j \sim \text{Bernoulli}(\pi_j)$ with $\pi_j = (0.005, 0.02, 0.05, 0.20)$.

Events times were simulated under the following time-constant additive hazards model:

$$\lambda(t|x) = \lambda_0(t) + \beta_1(t)X_1 + \beta_2(t)X_2 + \beta_3(t)X_3 + \beta_4(t)X_4$$

with $\lambda_0(t) = 0.05$ (constant) and $\beta_j(t) = (0.06, 0.02, 0.005, 0)$.

Independent censoring times were generated as $C \sim \text{Uniform}(0, c_{\max})$. The parameter $c_{\max}$ was calibrated using a large Monte Carlo sample from the data-generating mechanism to achieve an expected censoring proportion of approximately 20%, and the calibrated value was then fixed across all simulation replicates.

```
p <- 4
sigma <- diag(x = 1, nrow = p, ncol = p)
# prev <- c(0.005, 0.02, 0.05, 0.20)
prev <- runif(p, 0.2, 0.4)
beta <- c(0.06, 0.02, 0.005, 0)
lambda0 <- 0.05
target_cens <- 0.2
c_max <- calibrate_c_max_aalen(lambda0      = lambda0,
                               beta         = beta,
                               prev         = prev,
                               sigma        = sigma,
                               target_cens  = target_cens,
                               seed         = 2026)
c_max
```

```
[1] 72.64886
```

```
data_scen1 <- simulate_aalen_data(n, prev, sigma, lambda0, beta, c_max)
head(data_scen1)
```

	time	event	X1	X2	X3	X4
1	15.798980	1	0	0	0	1
2	16.570986	1	0	1	0	1
3	1.454993	1	0	0	0	0
4	21.854079	1	1	1	0	0
5	25.834806	1	0	0	0	0
6	26.417640	1	0	0	1	0

```
# Check prevalences
round(colMeans(data_scen1 %>% dplyr::select(starts_with("X"))), 3)
```

	X1	X2	X3	X4
	0.362	0.350	0.232	0.264

```
# Check censoring rate
mean(1 - data_scen1$event)
```

```
[1] 0.16
```

```
aalen_fit <- timereg::aalen(Surv(time, event)~X1+X2+X3+X4, data = data_scen1)
summary(aalen_fit)
```

Additive Aalen Model

Test for nonparametric terms

Test for non-significant effects

	Supremum-test of significance	p-value	H ₀ : B(t)=0
(Intercept)	9.29		0.000
X1	5.50		0.000
X2	2.77		0.112
X3	1.81		0.753
X4	1.68		0.790

Test for time invariant effects

	Kolmogorov-Smirnov test	p-value	H ₀ :constant effect
(Intercept)	0.830		0.000
X1	0.983		0.002
X2	0.450		0.151
X3	0.489		0.330
X4	0.223		0.899

	Cramer von Mises test	p-value	H ₀ :constant effect
(Intercept)	9.490		0.000
X1	10.100		0.002
X2	1.740		0.134
X3	1.700		0.396
X4	0.342		0.866

Call:

```
timereg::aalen(formula = Surv(time, event) ~ X1 + X2 + X3 + X4,  
  data = data_scen1)
```

```
time <- data_scen1$time  
event <- data_scen1$event  
X <- data_scen1 %>% dplyr::select(starts_with("X"))
```

1.3 Variable importance

We compare the following approaches for estimating the nuisance functions required for VIM computation:

- Aalen additive hazards model (`timereg::aalen`)
- Super Learner for survival prediction (`survivalSL::survivalSL`)
- Global survival stacking (`survML::stackG`)

We define time horizons, maximum follow-up time and times at which to approximate integrals.

```
landmark_times <- quantile(time, c(0.25, 0.75))  
approx_times <- sort(unique(c(0, time[event == 1], landmark_times)))  
approx_times <- approx_times[approx_times <= max(landmark_times)]  
length(approx_times)
```

[1] 322

We generated cross-fitting and sample-splitting folds.

```
folds <- survML::generate_folds(n, V, sample_split)
cf_folds <- folds$cf_folds
table(cf_folds)
```

```
cf_folds
 1  2  3  4  5  6  7  8  9 10
50 50 50 50 50 50 50 50 50 50
```

```
ss_folds <- folds$ss_folds
table(ss_folds)
```

```
ss_folds
 0   1
250 250
```

1.3.1 Global survival stacking

```
V0_stackG_preds_scen1 <- CV_generate_full_predictions_landmark(
  time = time,
  event = event,
  X = X,
  landmark_times = landmark_times,
  approx_times = approx_times,
  nuisance = "stackG",
  cf_folds = cf_folds)
str(V0_stackG_preds_scen1)
```

List of 6

```
$ CV_full_preds_train:List of 10
..$ : num [1:450, 1:2] 0.989 0.988 0.989 0.976 0.989 ...
..$ : num [1:450, 1:2] 0.985 0.986 0.986 0.978 0.986 ...
..$ : num [1:450, 1:2] 0.988 0.989 0.989 0.979 0.989 ...
..$ : num [1:450, 1:2] 0.989 0.989 0.989 0.979 0.989 ...
..$ : num [1:450, 1:2] 0.991 0.99 0.99 0.991 0.977 ...
..$ : num [1:450, 1:2] 0.989 0.989 0.989 0.979 0.989 ...
..$ : num [1:450, 1:2] 0.99 0.99 0.991 0.979 0.991 ...
..$ : num [1:450, 1:2] 0.987 0.988 0.976 0.988 0.988 ...
```

```

..$ : num [1:450, 1:2] 0.989 0.988 0.99 0.976 0.99 ...
..$ : num [1:450, 1:2] 0.986 0.986 0.978 0.987 0.988 ...
$ CV_full_preds      :List of 10
..$ : num [1:50, 1:2] 0.978 0.989 0.989 0.977 0.978 ...
..$ : num [1:50, 1:2] 0.986 0.975 0.986 0.986 0.985 ...
..$ : num [1:50, 1:2] 0.977 0.988 0.989 0.991 0.99 ...
..$ : num [1:50, 1:2] 0.979 0.989 0.979 0.989 0.989 ...
..$ : num [1:50, 1:2] 0.991 0.976 0.99 0.976 0.992 ...
..$ : num [1:50, 1:2] 0.989 0.989 0.979 0.979 0.989 ...
..$ : num [1:50, 1:2] 0.991 0.991 0.98 0.991 0.991 ...
..$ : num [1:50, 1:2] 0.987 0.978 0.988 0.987 0.987 ...
..$ : num [1:50, 1:2] 0.977 0.989 0.98 0.978 0.989 ...
..$ : num [1:50, 1:2] 0.987 0.977 0.986 0.987 0.987 ...
$ CV_S_preds        :List of 10
..$ : num [1:50, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:50, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:50, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:50, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:50, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:50, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:50, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:50, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:50, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:50, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
$ CV_S_preds_train   :List of 10
..$ : num [1:450, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:450, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:450, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:450, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:450, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:450, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:450, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:450, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:450, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:450, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
$ CV_G_preds         :List of 10
..$ : num [1:50, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:50, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:50, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:50, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:50, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:50, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:50, 1:322] 1 1 1 1 1 1 1 1 1 1 ...

```

```

..$ : num [1:50, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:50, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:50, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
$ CV_G_preds_train :List of 10
..$ : num [1:450, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:450, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:450, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:450, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:450, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:450, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:450, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:450, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:450, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:450, 1:322] 1 1 1 1 1 1 1 1 1 1 ...

```

```

CV_full_preds <- V0_stackG_preds_scen1$CV_full_preds
CV_full_preds_train <- V0_stackG_preds_scen1$CV_full_preds_train
CV_S_preds <- V0_stackG_preds_scen1$CV_S_preds
CV_G_preds <- V0_stackG_preds_scen1$CV_G_preds

indx <- 1:ncol(X)
vim_stackG_scen1 <- purrr::map_dfr(indx, function(indx_i) {

  char_indx <- as.character(indx_i)
  indx <- as.numeric(strsplit(char_indx, ",")[[1]])

  CV_reduced_preds <- CV_generate_reduced_predictions_landmark(
    time = time,
    event = event,
    X = X,
    landmark_times = landmark_times,
    cf_folds = cf_folds,
    indx = indx,
    full_preds_train = CV_full_preds_train
  )

  purrr::map_dfr(vims, function(vim) {

    output <- switch(
      vim,
      brier = survML::vim_brier(
        time = time,

```

```

    event = event,
    approx_times = approx_times,
    landmark_times = landmark_times,
    f_hat = CV_full_preds,
    fs_hat = CV_reduced_preds,
    S_hat = CV_S_preds,
    G_hat = CV_G_preds,
    cf_folds = cf_folds,
    sample_split = sample_split,
    ss_folds = ss_folds
  ),
  AUC = survML::vim_AUC(
    time = time,
    event = event,
    approx_times = approx_times,
    landmark_times = landmark_times,
    f_hat = purrr::map(CV_full_preds, ~ 1 - .x),
    fs_hat = purrr::map(CV_reduced_preds, ~ 1 - .x),
    S_hat = CV_S_preds,
    G_hat = CV_G_preds,
    cf_folds = cf_folds,
    sample_split = sample_split,
    ss_folds = ss_folds
  )
)

output %>%
  dplyr::mutate(
    vim = vim,
    indx = char_indx
  )
})
})

kable(vim_stackG_scen1)

```

	landmark_time	var_est	cil	ciu	cil_1sided	p	large_predictive	small_predictive	time_indx
25%...	12.801019	0.079442	0.73482352	- 1.9069368	- 0.466051	0.5349973	0.4555553	AUC 1	
			1.7480527		1.4542399				
75%...	216.914183	3.197514	10.8016003	- 0.9893215	- 0.312447	0.7268648	0.5293454	AUC 1	
			0.5942828		0.4669820				

	landmark_time	var_est	cil	ciu	cil_1sided	p	large_predictions	small_predictions	time	metric
25%...	32.801019	0.062098	0.03906489	-	0.1395754	-	0.0580987	-	-	brier 1
			0.0153781		0.0029219		0.7320562		0.7941548	
75%...	416.914183	-	1.2572806	-	0.1374359	-	0.5087608	-	-	brier 1
			0.0015575		0.1405509		0.1182044		0.5461880	0.5446306
25%...	52.801019	-	160.4003927	-	1.5586567	-	0.5056105	-	0.5349973	0.5462725
			0.0112751		1.5812070		1.3288035			AUC 2
75%...	616.914183	-	1.1029532	-	0.4291704	-	0.8511294	-	0.3936927	0.6239096
					0.6452190		0.5249324			AUC 2
25%...	72.801019	0.062424	0.03907565	-	0.1399117	-	0.0571728	-	-	brier 2
			0.0150631		0.0026052		0.7320562		0.7944805	
75%...	816.914183	-	0.0044098	-	0.2505803	-	0.1430324	-	0.4751421	-
					0.1342127		0.1119259		0.5461880	0.5505979
25%...	92.801019	-	159.7230842	-	1.5651741	-	0.5007105	-	0.5349973	0.5364370
			0.0014396		1.5680534		1.3161833			AUC 3
75%...	106.914183	-	0.0458836	-	0.2817905	-	0.7925438	-	0.4520617	0.6809803
					0.7007747		0.5807316			AUC 3
25%...	12.801019	0.062438	0.03907936	-	0.1399248	-	0.0571541	-	-	brier 3
			0.0150574		0.0025988		0.7320562		0.7944899	
75%...	126.914183	-	0.0076932	-	0.2528626	-	0.1464422	-	0.4567307	-
					0.1310558		0.1087486		0.5461880	0.5538812
25%...	13.801019	0.015116	0.0672667710	-	1.6182989	-	0.4926277	-	0.5349973	0.5198808
					1.5880658		1.3303165			AUC 4
75%...	146.914183	-	0.0542235	-	0.6005548	-	0.7938382	-	0.4428737	0.6726429
					0.6853944		0.5664836			AUC 4
25%...	13.801019	0.062549	0.03908419	-	0.1400451	-	0.0568311	-	-	brier 4
			0.0149467		0.0024874		0.7320562		0.7946054	
75%...	166.914183	-	0.0057744	-	0.2516847	-	0.1444582	-	0.4674794	-
					0.1329093		0.1106126		0.5461880	0.5519625

1.3.2 Aalen model

```
V0_aalen_preds_scen1 <- CV_generate_full_predictions_landmark(
  time = time,
  event = event,
  X = X,
  landmark_times = landmark_times,
  approx_times = approx_times,
  nuisance = "aalen",
```

```
cf_folds = cf_folds)
str(V0_aalen_preds_scen1)
```

List of 6

```
$ CV_full_preds_train:List of 10
..$ : num [1:450, 1:2] 0.793 0.757 0.854 0.719 0.854 ...
.. ..- attr(*, "dimnames")=List of 2
.. .. ..$ : chr [1:450] "1" "2" "3" "4" ...
.. .. ..$ : chr [1:2] "TP95" "TP287"
..$ : num [1:450, 1:2] 0.782 0.785 0.845 0.75 0.845 ...
.. ..- attr(*, "dimnames")=List of 2
.. .. ..$ : chr [1:450] "1" "2" "3" "4" ...
.. .. ..$ : chr [1:2] "TP92" "TP288"
..$ : num [1:450, 1:2] 0.771 0.768 0.828 0.744 0.828 ...
.. ..- attr(*, "dimnames")=List of 2
.. .. ..$ : chr [1:450] "1" "2" "3" "4" ...
.. .. ..$ : chr [1:2] "TP96" "TP287"
..$ : num [1:450, 1:2] 0.784 0.764 0.837 0.715 0.837 ...
.. ..- attr(*, "dimnames")=List of 2
.. .. ..$ : chr [1:450] "1" "2" "3" "4" ...
.. .. ..$ : chr [1:2] "TP98" "TP289"
..$ : num [1:450, 1:2] 0.772 0.84 0.84 0.885 0.738 ...
.. ..- attr(*, "dimnames")=List of 2
.. .. ..$ : chr [1:450] "1" "2" "3" "4" ...
.. .. ..$ : chr [1:2] "TP93" "TP287"
..$ : num [1:450, 1:2] 0.762 0.753 0.836 0.729 0.836 ...
.. ..- attr(*, "dimnames")=List of 2
.. .. ..$ : chr [1:450] "1" "2" "3" "4" ...
.. .. ..$ : chr [1:2] "TP100" "TP283"
..$ : num [1:450, 1:2] 0.803 0.776 0.861 0.707 0.849 ...
.. ..- attr(*, "dimnames")=List of 2
.. .. ..$ : chr [1:450] "1" "2" "3" "4" ...
.. .. ..$ : chr [1:2] "TP96" "TP288"
..$ : num [1:450, 1:2] 0.771 0.832 0.718 0.832 0.835 ...
.. ..- attr(*, "dimnames")=List of 2
.. .. ..$ : chr [1:450] "1" "2" "3" "4" ...
.. .. ..$ : chr [1:2] "TP101" "TP296"
..$ : num [1:450, 1:2] 0.774 0.738 0.861 0.7 0.861 ...
.. ..- attr(*, "dimnames")=List of 2
.. .. ..$ : chr [1:450] "1" "2" "3" "4" ...
.. .. ..$ : chr [1:2] "TP98" "TP290"
..$ : num [1:450, 1:2] 0.782 0.775 0.735 0.842 0.838 ...
```

```

.. ..- attr(*, "dimnames")=List of 2
.. .. ..$ : chr [1:450] "1" "2" "3" "4" ...
.. .. ..$ : chr [1:2] "TP95" "TP286"
$ CV_full_preds      :List of 10
..$ : num [1:50, 1:2] 0.754 0.854 0.854 0.7 0.754 ...
.. ..- attr(*, "dimnames")=List of 2
.. .. ..$ : chr [1:50] "1" "2" "3" "4" ...
.. .. ..$ : chr [1:2] "TP95" "TP287"
..$ : num [1:50, 1:2] 0.849 0.75 0.849 0.849 0.852 ...
.. ..- attr(*, "dimnames")=List of 2
.. .. ..$ : chr [1:50] "1" "2" "3" "4" ...
.. .. ..$ : chr [1:2] "TP92" "TP288"
..$ : num [1:50, 1:2] 0.747 0.771 0.786 0.841 0.844 ...
.. ..- attr(*, "dimnames")=List of 2
.. .. ..$ : chr [1:50] "1" "2" "3" "4" ...
.. .. ..$ : chr [1:2] "TP96" "TP287"
..$ : num [1:50, 1:2] 0.715 0.795 0.734 0.837 0.784 ...
.. ..- attr(*, "dimnames")=List of 2
.. .. ..$ : chr [1:50] "1" "2" "3" "4" ...
.. .. ..$ : chr [1:2] "TP98" "TP289"
..$ : num [1:50, 1:2] 0.803 0.709 0.84 0.709 0.813 ...
.. ..- attr(*, "dimnames")=List of 2
.. .. ..$ : chr [1:50] "1" "2" "3" "4" ...
.. .. ..$ : chr [1:2] "TP93" "TP287"
..$ : num [1:50, 1:2] 0.753 0.836 0.729 0.737 0.762 ...
.. ..- attr(*, "dimnames")=List of 2
.. .. ..$ : chr [1:50] "1" "2" "3" "4" ...
.. .. ..$ : chr [1:2] "TP100" "TP283"
..$ : num [1:50, 1:2] 0.861 0.861 0.731 0.861 0.849 ...
.. ..- attr(*, "dimnames")=List of 2
.. .. ..$ : chr [1:50] "1" "2" "3" "4" ...
.. .. ..$ : chr [1:2] "TP96" "TP288"
..$ : num [1:50, 1:2] 0.747 0.741 0.832 0.771 0.807 ...
.. ..- attr(*, "dimnames")=List of 2
.. .. ..$ : chr [1:50] "1" "2" "3" "4" ...
.. .. ..$ : chr [1:2] "TP101" "TP296"
..$ : num [1:50, 1:2] 0.704 0.774 0.738 0.734 0.774 ...
.. ..- attr(*, "dimnames")=List of 2
.. .. ..$ : chr [1:50] "1" "2" "3" "4" ...
.. .. ..$ : chr [1:2] "TP98" "TP290"
..$ : num [1:50, 1:2] 0.842 0.742 0.775 0.842 0.842 ...
.. ..- attr(*, "dimnames")=List of 2
.. .. ..$ : chr [1:50] "1" "2" "3" "4" ...

```

```

.. .. ..$ : chr [1:2] "TP95" "TP286"
$ CV_S_preds      :List of 10
..$ : num [1:50, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
.. ..- attr(*, "dimnames")=List of 2
.. .. ..$ : chr [1:50] "1" "2" "3" "4" ...
.. .. ..$ : chr [1:322] "TP1" "TP2" "TP3" "TP4" ...
..$ : num [1:50, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
.. ..- attr(*, "dimnames")=List of 2
.. .. ..$ : chr [1:50] "1" "2" "3" "4" ...
.. .. ..$ : chr [1:322] "TP1" "TP2" "TP3" "TP3" ...
..$ : num [1:50, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
.. ..- attr(*, "dimnames")=List of 2
.. .. ..$ : chr [1:50] "1" "2" "3" "4" ...
.. .. ..$ : chr [1:322] "TP1" "TP2" "TP3" "TP4" ...
..$ : num [1:50, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
.. ..- attr(*, "dimnames")=List of 2
.. .. ..$ : chr [1:50] "1" "2" "3" "4" ...
.. .. ..$ : chr [1:322] "TP1" "TP2" "TP3" "TP4" ...
..$ : num [1:50, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
.. ..- attr(*, "dimnames")=List of 2
.. .. ..$ : chr [1:50] "1" "2" "3" "4" ...
.. .. ..$ : chr [1:322] "TP1" "TP1" "TP2" "TP3" ...
..$ : num [1:50, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
.. ..- attr(*, "dimnames")=List of 2
.. .. ..$ : chr [1:50] "1" "2" "3" "4" ...
.. .. ..$ : chr [1:322] "TP1" "TP2" "TP3" "TP4" ...
..$ : num [1:50, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
.. ..- attr(*, "dimnames")=List of 2
.. .. ..$ : chr [1:50] "1" "2" "3" "4" ...
.. .. ..$ : chr [1:322] "TP1" "TP2" "TP3" "TP4" ...
..$ : num [1:50, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
.. ..- attr(*, "dimnames")=List of 2
.. .. ..$ : chr [1:50] "1" "2" "3" "4" ...
.. .. ..$ : chr [1:322] "TP1" "TP2" "TP3" "TP4" ...
..$ : num [1:50, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
.. ..- attr(*, "dimnames")=List of 2
.. .. ..$ : chr [1:50] "1" "2" "3" "4" ...
.. .. ..$ : chr [1:322] "TP1" "TP2" "TP2" "TP3" ...
$ CV_S_preds_train :List of 10

```

```

..$ : num [1:450, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
.. ..- attr(*, "dimnames")=List of 2
.. .. ..$ : chr [1:450] "1" "2" "3" "4" ...
.. .. ..$ : chr [1:322] "TP1" "TP2" "TP3" "TP4" ...
..$ : num [1:450, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
.. ..- attr(*, "dimnames")=List of 2
.. .. ..$ : chr [1:450] "1" "2" "3" "4" ...
.. .. ..$ : chr [1:322] "TP1" "TP2" "TP3" "TP3" ...
..$ : num [1:450, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
.. ..- attr(*, "dimnames")=List of 2
.. .. ..$ : chr [1:450] "1" "2" "3" "4" ...
.. .. ..$ : chr [1:322] "TP1" "TP2" "TP3" "TP4" ...
..$ : num [1:450, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
.. ..- attr(*, "dimnames")=List of 2
.. .. ..$ : chr [1:450] "1" "2" "3" "4" ...
.. .. ..$ : chr [1:322] "TP1" "TP2" "TP3" "TP4" ...
..$ : num [1:450, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
.. ..- attr(*, "dimnames")=List of 2
.. .. ..$ : chr [1:450] "1" "2" "3" "4" ...
.. .. ..$ : chr [1:322] "TP1" "TP1" "TP2" "TP3" ...
..$ : num [1:450, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
.. ..- attr(*, "dimnames")=List of 2
.. .. ..$ : chr [1:450] "1" "2" "3" "4" ...
.. .. ..$ : chr [1:322] "TP1" "TP2" "TP3" "TP4" ...
..$ : num [1:450, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
.. ..- attr(*, "dimnames")=List of 2
.. .. ..$ : chr [1:450] "1" "2" "3" "4" ...
.. .. ..$ : chr [1:322] "TP1" "TP2" "TP3" "TP4" ...
..$ : num [1:450, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
.. ..- attr(*, "dimnames")=List of 2
.. .. ..$ : chr [1:450] "1" "2" "3" "4" ...
.. .. ..$ : chr [1:322] "TP1" "TP2" "TP2" "TP3" ...
$ CV_G_preds :List of 10
..$ : num [1:50, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
.. ..- attr(*, "dimnames")=List of 2

```

```

.. .. ..$ : chr [1:50] "1" "2" "3" "4" ...
.. .. ..$ : chr [1:322] "TP1" "TP1" "TP1" "TP1" ...
..$ : num [1:50, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
.. ..- attr(*, "dimnames")=List of 2
.. .. ..$ : chr [1:50] "1" "2" "3" "4" ...
.. .. ..$ : chr [1:322] "TP1" "TP1" "TP1" "TP1" ...
..$ : num [1:50, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
.. ..- attr(*, "dimnames")=List of 2
.. .. ..$ : chr [1:50] "1" "2" "3" "4" ...
.. .. ..$ : chr [1:322] "TP1" "TP1" "TP1" "TP1" ...
..$ : num [1:50, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
.. ..- attr(*, "dimnames")=List of 2
.. .. ..$ : chr [1:50] "1" "2" "3" "4" ...
.. .. ..$ : chr [1:322] "TP1" "TP1" "TP1" "TP1" ...
..$ : num [1:50, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
.. ..- attr(*, "dimnames")=List of 2
.. .. ..$ : chr [1:50] "1" "2" "3" "4" ...
.. .. ..$ : chr [1:322] "TP1" "TP1" "TP1" "TP1" ...
..$ : num [1:50, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
.. ..- attr(*, "dimnames")=List of 2
.. .. ..$ : chr [1:50] "1" "2" "3" "4" ...
.. .. ..$ : chr [1:322] "TP1" "TP1" "TP1" "TP1" ...
..$ : num [1:50, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
.. ..- attr(*, "dimnames")=List of 2
.. .. ..$ : chr [1:50] "1" "2" "3" "4" ...
.. .. ..$ : chr [1:322] "TP1" "TP1" "TP1" "TP1" ...
..$ : num [1:50, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
.. ..- attr(*, "dimnames")=List of 2
.. .. ..$ : chr [1:50] "1" "2" "3" "4" ...
.. .. ..$ : chr [1:322] "TP1" "TP1" "TP1" "TP1" ...
$ CV_G_preds_train :List of 10
..$ : num [1:450, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
.. ..- attr(*, "dimnames")=List of 2
.. .. ..$ : chr [1:450] "1" "2" "3" "4" ...
.. .. ..$ : chr [1:322] "TP1" "TP1" "TP1" "TP1" ...

```

```

..$ : num [1:450, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
.. ..- attr(*, "dimnames")=List of 2
.. .. ..$ : chr [1:450] "1" "2" "3" "4" ...
.. .. ..$ : chr [1:322] "TP1" "TP1" "TP1" "TP1" ...
..$ : num [1:450, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
.. ..- attr(*, "dimnames")=List of 2
.. .. ..$ : chr [1:450] "1" "2" "3" "4" ...
.. .. ..$ : chr [1:322] "TP1" "TP1" "TP1" "TP1" ...
..$ : num [1:450, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
.. ..- attr(*, "dimnames")=List of 2
.. .. ..$ : chr [1:450] "1" "2" "3" "4" ...
.. .. ..$ : chr [1:322] "TP1" "TP1" "TP1" "TP1" ...
..$ : num [1:450, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
.. ..- attr(*, "dimnames")=List of 2
.. .. ..$ : chr [1:450] "1" "2" "3" "4" ...
.. .. ..$ : chr [1:322] "TP1" "TP1" "TP1" "TP1" ...
..$ : num [1:450, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
.. ..- attr(*, "dimnames")=List of 2
.. .. ..$ : chr [1:450] "1" "2" "3" "4" ...
.. .. ..$ : chr [1:322] "TP1" "TP1" "TP1" "TP1" ...
..$ : num [1:450, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
.. ..- attr(*, "dimnames")=List of 2
.. .. ..$ : chr [1:450] "1" "2" "3" "4" ...
.. .. ..$ : chr [1:322] "TP1" "TP1" "TP1" "TP1" ...
..$ : num [1:450, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
.. ..- attr(*, "dimnames")=List of 2
.. .. ..$ : chr [1:450] "1" "2" "3" "4" ...
.. .. ..$ : chr [1:322] "TP1" "TP1" "TP1" "TP1" ...
..$ : num [1:450, 1:322] 1 1 1 1 1 1 1 1 1 1 ...
.. ..- attr(*, "dimnames")=List of 2
.. .. ..$ : chr [1:450] "1" "2" "3" "4" ...
.. .. ..$ : chr [1:322] "TP1" "TP1" "TP1" "TP1" ...

```

```

CV_full_preds <- V0_aalen_preds_scen1$CV_full_preds
CV_full_preds_train <- V0_aalen_preds_scen1$CV_full_preds_train
CV_S_preds <- V0_aalen_preds_scen1$CV_S_preds
CV_G_preds <- V0_aalen_preds_scen1$CV_G_preds

```

```

indx_s <- 1:ncol(X)
vim_aalen_scen1 <- purrr::map_dfr(indx_s, function(indx_i) {

  char_indx <- as.character(indx_i)
  indx <- as.numeric(strsplit(char_indx, split = ",")[[1]])

  CV_reduced_preds <- CV_generate_reduced_predictions_landmark(
    time = time,
    event = event,
    X = X,
    landmark_times = landmark_times,
    cf_folds = cf_folds,
    indx = indx,
    full_preds_train = CV_full_preds_train
  )

  purrr::map_dfr(vims, function(vim) {

    output <- switch(
      vim,
      brier = survML::vim_brier(
        time = time,
        event = event,
        approx_times = approx_times,
        landmark_times = landmark_times,
        f_hat = CV_full_preds,
        fs_hat = CV_reduced_preds,
        S_hat = CV_S_preds,
        G_hat = CV_G_preds,
        cf_folds = cf_folds,
        sample_split = sample_split,
        ss_folds = ss_folds
      ),
      AUC = survML::vim_AUC(
        time = time,
        event = event,
        approx_times = approx_times,
        landmark_times = landmark_times,
        f_hat = purrr::map(CV_full_preds, ~ 1 - .x),
        fs_hat = purrr::map(CV_reduced_preds, ~ 1 - .x),
        S_hat = CV_S_preds,
        G_hat = CV_G_preds,

```



```

        cf_folds = cf_folds,
        sample_split = sample_split,
        ss_folds = ss_folds
    )
)

output %>%
  dplyr::mutate(
    vim = vim,
    indx = char_indx
  )
})
})

kable(vim_aalen_scen1)

```

	landmark	time	var_est	cil	ciu	cil_1sided	p	large_predictives	smaller_predictives	time_indx
25%...	12.801019	0.1274856	0.096476	0.092591	0.1245712	0.1028266	0.8017280	0.5981204	0.4706348	AUC 1
75%...	216.914183	0.1364164	0.464609	0.293183	0.2435144	0.465368	0.0062701	0.6660939	0.5296776	AUC 1
25%...	32.801019	0.0214903	0.1207354	-	0.0645663	-	0.1640169	-	-	brier 1
					0.0215778		0.0146529	0.4867229	0.5082172	
75%...	416.914183	-	0.0990282	-	-	-	0.9978930	-	-	brier 1
			0.0569550	0.0959633	0.179466	0.0896918	0.4102502	0.3532952		
25%...	52.801019	0.0676906	0.168179	-	0.1863821	-	0.1318304	0.5981204	0.5304298	AUC 2
					0.0510010		0.0319185			
75%...	616.914183	0.0240006	0.7344700	-	0.1302440	-	0.3288906	0.6660939	0.6420843	AUC 2
					0.0822248		0.0651451			
25%...	72.801019	0.0260082	0.1254260	-	0.0699090	-	0.1227910	-	-	brier 2
					0.0178925		0.0108344	0.4867229	0.5127312	
75%...	816.914183	-	0.1242255	-	0.0284885	-	0.7523666	-	-	brier 2
			0.0152017	0.0588918	0.0518676	0.4102502	0.3950485			
25%...	92.801019	0.0500195	0.147529	-	0.1685772	-	0.2041406	0.5981204	0.5481009	AUC 3
					0.0685383		0.0494773			
75%...	106.914183	0.0200873	0.6948519	-	0.1234167	-	0.3515953	0.6660939	0.6460067	AUC 3
					0.0832422		0.0666296			
25%...	12.801019	0.0260339	0.1257884	-	0.0699981	-	0.1228984	-	-	brier 3
					0.0179302		0.0108619	0.4867229	0.5127569	
75%...	126.914183	-	0.1263108	-	0.0316346	-	0.7097245	-	-	brier 3
			0.0124207	0.0564760	0.0493931	0.4102502	0.3978295			
25%...	13.801019	0.0696985	0.689043	-	0.1917150	-	0.1314407	0.5981204	0.5284218	AUC 4
					0.0523179		0.0327009			

	landmark_time	var_est	cil	ciu	cil_1sided	p	large_prediction	small_prediction	time_index
75%...	146.914183	0.0129786	116583	-	0.1175452	-	0.4039403	0.6660939	0.6531204 AUC 4
			0.0915981		0.0747857				
25%...	13.801019	0.0268403	1248647	-	0.0706437	-	0.1148701	-	- brier 4
			0.0169611		0.0099189		0.4867229	0.5135642	
75%...	166.914183	-	0.1257263	-	0.0313096	-	0.7135566	-	- brier 4
		0.0126436	0.0565969		0.0495304		0.4102502	0.3976066	

2 Scenario 2: Mixed additive and multiplicative hazards

2.1 Objectives

This scenario:

- evaluates the overall performance of the VIM procedure under mixed additive and multiplicative hazards: bias, confidence interval coverage, confidence interval width, rejection probability
- assesses the robustness of the VIM procedure to nuisance-model choice required for VIM computation: Cox-Aalen model (`timereg::cox.aalen`), survival Super Learner (`survivalSL::survivalSL`), global survival stacking (`survML::stackG`)

2.2 Data-generating mechanism

The covariates are generated *independently* as follows:

- $X = (X_1, X_2, X_3, X_4)$
- $X_1, X_2 \sim \mathcal{B}(0.4)$
- $X_3, X_4 \sim \mathcal{N}(0, 1)$

To generate mixed additive and multiplicative effects, event times were simulated under a Cox-Aalen model with constant baseline hazard: $\lambda_0(t) = 0.05$. Additive effects were assigned to X_1 and X_3 with coefficients $\beta_1(t) = 0.06$ and $\beta_3(t) = 0.005$, respectively. Multiplicative (proportional hazards) effects were assigned to X_2 and X_4 with coefficients $\beta_2 = 0.5$ and $\beta_4 = 0.2$, corresponding to hazard ratios of approximately 1.65 and 1.22.

Independent censoring times were generated as $C \sim \text{Uniform}(0, c_{\max})$. The parameter $c_{\max}$ was calibrated using a large Monte Carlo sample from the data-generating mechanism to achieve an expected censoring proportion of approximately 20%, and the calibrated value was then fixed across all simulation replicates.

```

prev_add <- c(0.4, 0.4)
sigma_add <- diag(x = 1, nrow = 2, ncol = 2)
beta_add <- c(0.06, 0.005)
prev_mult <- 0.4
sigma_mult_bin <- diag(x = 1, nrow = 1, ncol = 1)
mean_mult <- 0
sigma_mult_cont <- diag(x = 1, nrow = 1, ncol = 1)
beta_mult <- c(0.5, 0.2)
lambda0 <- 0.05
target_cens <- 0.2
c_max <- calibrate_c_max_cox_aalen(lambda0      = lambda0,
                                   beta_add      = beta_add,
                                   beta_mult     = beta_mult,
                                   prev_add      = prev_add,
                                   sigma_add     = sigma_add,
                                   prev_mult     = prev_mult,
                                   mean_mult     = mean_mult,
                                   sigma_mult_bin = sigma_mult_bin,
                                   sigma_mult_cont = sigma_mult_cont,
                                   target_cens    = target_cens,
                                   seed          = 2026)
c_max

```

```
[1] 63.50589
```

```

data_scen2 <- simulate_cox_aalen_data(
  n, lambda0, beta_add, beta_mult,
  prev_add, sigma_add, prev_mult, mean_mult,
  sigma_mult_bin, sigma_mult_cont, c_max
)
head(data_scen2)

```

	time	event	X1	X2	X3	X4
1	14.799082	1	0	1	0	-0.14964990
2	3.499728	1	1	1	1	2.79265615
3	1.312034	1	0	1	0	0.04056139
4	21.180384	1	1	0	1	-1.50816900
5	10.240640	1	1	1	0	0.46224676
6	27.884512	0	0	0	0	-0.23424115

```
# Check prevalences
colMeans(data_scen2 %>% dplyr::select(X1:X3))
```

```
      X1      X2      X3
0.418 0.402 0.426
```

```
# Check mean and sd
round(mean(data_scen2$X4), 3)
```

```
[1] 0
```

```
sd(data_scen2$X4)
```

```
[1] 1
```

```
# Check censoring rate
mean(1 - data_scen2$event)
```

```
[1] 0.168
```

```
cox_aalen_fit <- timereg::cox.aalen(Surv(time, event)~X1+X2+prop(X3)+prop(X4),
                                   data = data_scen2, n.sim = 1000, basesim = 1)
summary(cox_aalen_fit)
```

Cox-Aalen Model

Test for Aalen terms

Test for nonparametric terms

Test for non-significant effects

	Supremum-test of significance	p-value	H ₀ : B(t)=0
(Intercept)	8.92	0.000	
X1	5.21	0.000	
X2	2.78	0.117	

Test for time invariant effects

	Kolmogorov-Smirnov test	p-value	H ₀ :constant effect
(Intercept)	0.666	0.000	

X1	0.870	0.000
X2	0.611	0.122

Proportional Cox terms :

	Coef.	SE Robust	SE D2log(L) ⁻¹	z	P-val	lower2.5%	upper97.5%
prop(X3)	0.350	0.1030	0.0954	0.1010	3.67	2.41e-04	0.148
prop(X4)	0.235	0.0566	0.0498	0.0525	4.71	2.46e-06	0.124

Test of Proportionality

	sup hat U(t)	p-value H ₀
prop(X3)	9.49	0.224
prop(X4)	14.30	0.559

```
time <- data_scen2$time
event <- data_scen2$event
X <- data_scen2 %>% dplyr::select(starts_with("X"))
```

2.3 Variable importance

We compare the following approaches for estimating the nuisance functions required for VIM computation:

- Cox-Aalen model (`timereg::cox.aalen`)
- Survival Super Learner (`survivalSL::survivalSL`)
- Global survival stacking (`survML::stackG`)

We define time horizons, maximum follow-up time and times at which to approximate integrals.

```
landmark_times <- quantile(time, c(0.25, 0.75))
approx_times <- sort(unique(c(0, time[event == 1], landmark_times)))
approx_times <- approx_times[approx_times <= max(landmark_times)]
length(approx_times)
```

```
[1] 323
```

We generated cross-fitting and sample-splitting folds.

```
folds <- survML::generate_folds(n, V, sample_split)
cf_folds <- folds$cf_folds
table(cf_folds)
```

```
cf_folds
  1  2  3  4  5  6  7  8  9 10
50 50 50 50 50 50 50 50 50 50
```

```
ss_folds <- folds$ss_folds
table(ss_folds)
```

```
ss_folds
  0    1
250 250
```

2.3.1 Global survival stacking

```
V0_stackG_preds_scen2 <- CV_generate_full_predictions_landmark(
  time = time,
  event = event,
  X = X,
  landmark_times = landmark_times,
  approx_times = approx_times,
  nuisance = "stackG",
  cf_folds = cf_folds)
str(V0_stackG_preds_scen2)
```

List of 6

```
$ CV_full_preds_train:List of 10
..$ : num [1:450, 1:2] 0.99 0.927 0.99 0.979 0.976 ...
..$ : num [1:450, 1:2] 0.99 0.921 0.989 0.981 0.993 ...
..$ : num [1:450, 1:2] 0.988 0.931 0.988 0.983 0.977 ...
..$ : num [1:450, 1:2] 0.989 0.932 0.988 0.979 0.973 ...
..$ : num [1:450, 1:2] 0.993 0.928 0.982 0.981 0.995 ...
..$ : num [1:450, 1:2] 0.99 0.935 0.99 0.982 0.978 ...
..$ : num [1:450, 1:2] 0.992 0.922 0.991 0.979 0.994 ...
..$ : num [1:450, 1:2] 0.925 0.99 0.982 0.976 0.993 ...
..$ : num [1:450, 1:2] 0.989 0.914 0.989 0.981 0.973 ...
..$ : num [1:450, 1:2] 0.99 0.99 0.983 0.978 0.991 ...
$ CV_full_preds      :List of 10
..$ : num [1:50, 1:2] 0.992 0.992 0.987 0.988 0.977 ...
..$ : num [1:50, 1:2] 0.974 0.991 0.992 0.985 0.993 ...
..$ : num [1:50, 1:2] 0.987 0.982 0.99 0.989 0.982 ...
..$ : num [1:50, 1:2] 0.964 0.984 0.991 0.983 0.989 ...
```

```

..$ : num [1:50, 1:2] 0.994 0.995 0.981 0.981 0.973 ...
..$ : num [1:50, 1:2] 0.982 0.993 0.98 0.988 0.988 ...
..$ : num [1:50, 1:2] 0.98 0.987 0.988 0.981 0.977 ...
..$ : num [1:50, 1:2] 0.99 0.987 0.993 0.98 0.994 ...
..$ : num [1:50, 1:2] 0.979 0.985 0.991 0.964 0.979 ...
..$ : num [1:50, 1:2] 0.944 0.99 0.989 0.982 0.967 ...
$ CV_S_preds      :List of 10
..$ : num [1:50, 1:323] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:50, 1:323] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:50, 1:323] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:50, 1:323] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:50, 1:323] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:50, 1:323] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:50, 1:323] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:50, 1:323] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:50, 1:323] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:50, 1:323] 1 1 1 1 1 1 1 1 1 1 ...
$ CV_S_preds_train :List of 10
..$ : num [1:450, 1:323] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:450, 1:323] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:450, 1:323] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:450, 1:323] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:450, 1:323] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:450, 1:323] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:450, 1:323] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:450, 1:323] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:450, 1:323] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:450, 1:323] 1 1 1 1 1 1 1 1 1 1 ...
$ CV_G_preds      :List of 10
..$ : num [1:50, 1:323] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:50, 1:323] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:50, 1:323] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:50, 1:323] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:50, 1:323] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:50, 1:323] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:50, 1:323] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:50, 1:323] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:50, 1:323] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:50, 1:323] 1 1 1 1 1 1 1 1 1 1 ...
$ CV_G_preds_train :List of 10
..$ : num [1:450, 1:323] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:450, 1:323] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:450, 1:323] 1 1 1 1 1 1 1 1 1 1 ...

```

```

..$ : num [1:450, 1:323] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:450, 1:323] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:450, 1:323] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:450, 1:323] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:450, 1:323] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:450, 1:323] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:450, 1:323] 1 1 1 1 1 1 1 1 1 1 ...

```

```

CV_full_preds <- V0_stackG_preds_scen2$CV_full_preds
CV_full_preds_train <- V0_stackG_preds_scen2$CV_full_preds_train
CV_S_preds <- V0_stackG_preds_scen2$CV_S_preds
CV_G_preds <- V0_stackG_preds_scen2$CV_G_preds

indx <- 1:ncol(X)
vim_stackG_scen2 <- purrr::map_dfr(indx, function(indx_i) {

  char_indx <- as.character(indx_i)
  indx <- as.numeric(strsplit(char_indx, split = ",")[[1]])

  CV_reduced_preds <- CV_generate_reduced_predictions_landmark(
    time = time,
    event = event,
    X = X,
    landmark_times = landmark_times,
    cf_folds = cf_folds,
    indx = indx,
    full_preds_train = CV_full_preds_train
  )

  purrr::map_dfr(vims, function(vim) {

    output <- switch(
      vim,
      brier = survML::vim_brier(
        time = time,
        event = event,
        approx_times = approx_times,
        landmark_times = landmark_times,
        f_hat = CV_full_preds,
        fs_hat = CV_reduced_preds,
        S_hat = CV_S_preds,
        G_hat = CV_G_preds,

```



```

      cf_folds = cf_folds,
      sample_split = sample_split,
      ss_folds = ss_folds
    ),
    AUC = survML::vim_AUC(
      time = time,
      event = event,
      approx_times = approx_times,
      landmark_times = landmark_times,
      f_hat = purrr::map(CV_full_preds, ~ 1 - .x),
      fs_hat = purrr::map(CV_reduced_preds, ~ 1 - .x),
      S_hat = CV_S_preds,
      G_hat = CV_G_preds,
      cf_folds = cf_folds,
      sample_split = sample_split,
      ss_folds = ss_folds
    )
  )
}

output %>%
  dplyr::mutate(
    vim = vim,
    indx = char_indx
  )
})
})

kable(vim_stackG_scen2)

```

	landmark_time	var_est	cil	ciu	cil_1sided	p	large_predictive	smaller_predictive	time_index
25%...	12.426343	0.085650	22.398897	1 - 1.4570587	- 0.4512886	0.725040	0.5868539	AUC 1	
			1.2857583	1.0652721					
75%...	212.492344	0.060576	1.0805193	- 0.4732037	- 0.3867756	0.805640	0.6199878	AUC 1	
			0.3520513	0.2857117					
25%...	32.426343	0.026335	83925064	- 0.1039965	- 0.2531374	-	-	brier 1	
			0.0513250	0.0388392	0.7471726	0.7735084			
75%...	412.492344	- 0.5793101	- 0.0229598	- 0.9309631	-	-	-	brier 1	
		0.0713884	0.1657366	0.1505679	0.5746024	0.5032140			
25%...	52.426343	0.057837	47.9171218	- 1.4039039	- 0.4664427	0.725040	0.6146666	AUC 2	
			1.2882291	1.0718172					

	landmark_time	var_est	cil	ciu	cil_1sided	p	large_predictions	small_predictions	time	metric
75%...	612.492344	-	10.4619834	-	0.3843769	-	0.5322758	0.805640	0.6971323	AUC 2
		0.0165683		0.4175136		0.3530522				
25%...	72.426343	0.0270569	3927878	-	0.1047455	-	0.2474283	-	-	brier 2
				0.0506316		0.0381414		0.7471726	0.7742295	
75%...	812.492344	-	0.5745220	-	0.0293111	-	0.9112560	-	-	brier 2
		0.0646463		0.1586038		0.1434980		0.5746024	0.5099560	
25%...	92.426343	0.0608492	5.9677618	-	1.4521075	-	0.4658435	0.725040	0.6116549	AUC 3
				1.3304093		1.1067317				
75%...	102.492344	-	10.8161881	-	0.3736727	-	0.5649282	0.805640	0.7145673	AUC 3
		0.0340033		0.4416794		0.3761359				
25%...	12.426343	0.0270392	3926249	-	0.1047116	-	0.2475252	-	-	brier 3
				0.0506333		0.0381456		0.7471726	0.7742118	
75%...	122.492344	-	0.5608948	-	0.0265731	-	0.9190865	-	-	brier 3
		0.0662634		0.1590999		0.1441743		0.5746024	0.5083390	
25%...	13.426343	0.0828380	7.2793204	-	1.4252593	-	0.4518672	0.725040	0.5896660	AUC 4
				1.2595832		1.0437574				
75%...	142.492344	0.0605282	8.8563945	-	0.5049938	-	0.3947676	0.805640	0.6200353	AUC 4
				0.3839364		0.3124782				
25%...	15.426343	0.0267854	3925843	-	0.1044538	-	0.2495425	-	-	brier 4
				0.0508830		0.0383960		0.7471726	0.7739580	
75%...	162.492344	-	0.5731887	-	0.0252601	-	0.9239884	-	-	brier 4
		0.0685883		0.1624367		0.1473483		0.5746024	0.5060141	

2.3.2 Cox-Aalen model

```

V0_cox_aalen_preds_scen2 <- CV_generate_full_predictions_landmark(
  time = time,
  event = event,
  X = X,
  landmark_times = landmark_times,
  approx_times = approx_times,
  nuisance = "cox.aalen",
  cf_folds = cf_folds)
str(V0_cox_aalen_preds_scen2)

CV_full_preds <- V0_cox_aalen_preds_scen2$CV_full_preds
CV_full_preds_train <- V0_cox_aalen_preds_scen2$CV_full_preds_train
CV_S_preds <- V0_cox_aalen_preds_scen2$CV_S_preds
CV_G_preds <- V0_cox_aalen_preds_scen2$CV_G_preds

```

```

indx<= 1:ncol(X)
vim_cox_aalen_scen2 <- purrr::map_dfr(indxs, function(indx_i) {

  char_indx <- as.character(indx_i)
  indx <- as.numeric(strsplit(char_indx, split = ",")[[1]])

  CV_reduced_preds <- CV_generate_reduced_predictions_landmark(
    time = time,
    event = event,
    X = X,
    landmark_times = landmark_times,
    cf_folds = cf_folds,
    indx = indx,
    full_preds_train = CV_full_preds_train
  )

  purrr::map_dfr(vims, function(vim) {

    output <- switch(
      vim,
      brier = survML::vim_brier(
        time = time,
        event = event,
        approx_times = approx_times,
        landmark_times = landmark_times,
        f_hat = CV_full_preds,
        fs_hat = CV_reduced_preds,
        S_hat = CV_S_preds,
        G_hat = CV_G_preds,
        cf_folds = cf_folds,
        sample_split = sample_split,
        ss_folds = ss_folds
      ),
      AUC = survML::vim_AUC(
        time = time,
        event = event,
        approx_times = approx_times,
        landmark_times = landmark_times,
        f_hat = purrr::map(CV_full_preds, ~ 1 - .x),
        fs_hat = purrr::map(CV_reduced_preds, ~ 1 - .x),
        S_hat = CV_S_preds,
        G_hat = CV_G_preds,

```

```

        cf_folds = cf_folds,
        sample_split = sample_split,
        ss_folds = ss_folds
    )
)

output %>%
  dplyr::mutate(
    vim = vim,
    indx = char_indx
  )
})
})

kable(vim_cox_aalen_scen2)

```

3 Scenario 3: Rare genomic alterations with mixed effects under proportional

hazards

3.1 Objectives

This scenario aims to evaluate the overall performance of the VIM procedure to *rare alterations* exhibiting *heterogeneous effect sizes (ranging from weak to strong)* under a *proportional hazards model*. More specifically, the objective is to assess whether the VIM approach can:

- detect rare yet highly prognostic alterations,
- discriminate between weak and strong prognostic signals, and
- correctly identify non-informative (null) variables.

3.2 Data generating mechanism

All covariates were generated independently as binary variables:

$$X_j \sim \mathcal{B}(\pi_j)$$

with $\pi \in \{0.01, 0.1\}$ to reflect genomic rarity.

The event times follows a proportional hazards model with a with constant baseline hazard $\lambda_0(t) = 0.1$ such that

$$\lambda(t | X) = \lambda_0(t) \exp(X^\top \beta).$$

Covariates were partitioned according to both their prevalence and prognostic effect size, resulting in the following combinations:

```
knitr::kable(
  data.frame(
    `Alteration frequency` = c(
      rep("Very rare ( $\pi = 0.01$ )", 5),
      rep("Rare ( $\pi = 0.10$ )", 5)
    ),
    `Effect category` = c(
      "Strong", "Strong", "Moderate", "Weak", "Null",
      "Strong", "Strong", "Moderate", "Weak", "Null"
    ),
    `Hazard ratio` = c(
      3.0, 2.0, 1.5, 1.2, 1.0,
      3.0, 2.0, 1.5, 1.2, 1.0
    ),
    `Regression coefficient` = c(
      " $\log(3)$ ",
      " $\log(2)$ ",
      " $\log(1.5)$ ",
      " $\log(1.2)$ ",
      "0",
      " $\log(3)$ ",
      " $\log(2)$ ",
      " $\log(1.5)$ ",
      " $\log(1.2)$ ",
      "0"
    )
  ),
  escape = FALSE,
  align = "llcc"
)
```

Alteration.frequency	Effect.category	Hazard.ratio	Regression.coefficient
Very rare ($\pi = 0.01$)	Strong	3.0	$\log(3)$
Very rare ($\pi = 0.01$)	Strong	2.0	$\log(2)$
Very rare ($\pi = 0.01$)	Moderate	1.5	$\log(1.5)$
Very rare ($\pi = 0.01$)	Weak	1.2	$\log(1.2)$
Very rare ($\pi = 0.01$)	Null	1.0	0
Rare ($\pi = 0.10$)	Strong	3.0	$\log(3)$
Rare ($\pi = 0.10$)	Strong	2.0	$\log(2)$
Rare ($\pi = 0.10$)	Moderate	1.5	$\log(1.5)$
Rare ($\pi = 0.10$)	Weak	1.2	$\log(1.2)$
Rare ($\pi = 0.10$)	Null	1.0	0

This design allows us to assess whether the VIM procedure can detect rare but strongly prognostic alterations, distinguish weak from moderate and strong effects, and correctly assign low importance to null variables.

Independent censoring times were generated as $C \sim \text{Uniform}(0, c_{max})$. The parameter c_{max} was calibrated using a large Monte Carlo sample from the data-generating mechanism to achieve an expected censoring proportion of approximately 20%, and the calibrated value was then fixed across all simulation replicates.

```
p <- 10
prev <- c(rep(0.01, 5), rep(0.1, 5))
beta <- rep(c(log(3), log(2), log(1.5), log(1.2), 0), 2)
sigma <- diag(x = 1, nrow = p, ncol = p)
lambda0 <- 0.1
target_cens <- 0.2
c_max_scen3 <- calibrate_c_max_cox(lambda0 = lambda0,
                                   beta = beta,
                                   prev = prev,
                                   sigma = sigma,
                                   target_cens = target_cens,
                                   seed = 2026)
c_max_scen3
```

```
[1] 40.89694
```

```
data_scen3 <- simulate_cox_data(n, prev, sigma, lambda0, beta, c_max_scen3)
head(data_scen3)
```

	time	event	X1	X2	X3	X4	X5	X6	X7	X8	X9	X10
1	10.2834804	1	0	0	0	0	0	0	1	0	0	0
2	0.3443951	1	0	0	0	0	0	1	0	0	0	0
3	3.3076462	1	0	0	0	0	0	0	0	1	0	0
4	3.4743000	0	0	0	0	0	0	0	0	0	0	0
5	15.6715162	1	0	0	0	0	0	0	0	0	1	0
6	8.8225439	1	0	0	0	0	0	0	0	0	0	1

```
# Check empirical prevalences
round(colMeans(data_scen3 %>% dplyr::select(starts_with("X"))), 3)
```

	X1	X2	X3	X4	X5	X6	X7	X8	X9	X10
	0.006	0.012	0.008	0.006	0.010	0.102	0.098	0.118	0.096	0.104

```
# Check censoring
mean(1 - data_scen3$event)
```

```
[1] 0.192
```

```
# Cox PH model
coxph_fit <- coxph(Surv(time, event)~., data_scen3)
summary(coxph_fit)
```

Call:

```
coxph(formula = Surv(time, event) ~ ., data = data_scen3)
```

n= 500, number of events= 404

	coef	exp(coef)	se(coef)	z	Pr(> z)
X1	0.6478	1.9113	0.5820	1.113	0.265701
X2	0.7097	2.0334	0.4624	1.535	0.124821
X3	1.3408	3.8219	0.5910	2.269	0.023288 *
X4	0.8246	2.2809	0.5862	1.407	0.159541
X5	1.1361	3.1146	0.4534	2.506	0.012213 *
X6	1.4672	4.3370	0.1662	8.830	< 2e-16 ***
X7	0.5836	1.7925	0.1682	3.470	0.000521 ***
X8	0.2681	1.3075	0.1577	1.700	0.089156 .
X9	0.1631	1.1772	0.1627	1.002	0.316110
X10	-0.2531	0.7764	0.1712	-1.478	0.139330

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

	exp(coef)	exp(-coef)	lower .95	upper .95
X1	1.9113	0.5232	0.6108	5.980
X2	2.0334	0.4918	0.8215	5.033
X3	3.8219	0.2616	1.2001	12.171
X4	2.2809	0.4384	0.7230	7.196
X5	3.1146	0.3211	1.2809	7.574
X6	4.3370	0.2306	3.1315	6.007
X7	1.7925	0.5579	1.2891	2.493
X8	1.3075	0.7648	0.9598	1.781
X9	1.1772	0.8495	0.8557	1.619
X10	0.7764	1.2880	0.5551	1.086

Concordance= 0.609 (se = 0.014)

Likelihood ratio test= 79.78 on 10 df, p=6e-13

Wald test = 98.78 on 10 df, p=<2e-16

Score (logrank) test = 112.8 on 10 df, p=<2e-16

```
time <- data_scen3$time
event <- data_scen3$event
X <- data_scen3 %>% dplyr::select(starts_with("X"))
```

3.3 Variable importance

We define time horizons, maximum follow-up time and times at which to approximate integrals.

```
landmark_times <- quantile(time, c(0.25, 0.75))
approx_times <- sort(unique(c(0, time[event == 1], landmark_times)))
approx_times <- approx_times[approx_times <= max(landmark_times)]
length(approx_times)
```

```
[1] 317
```

We generated cross-fitting and sample-splitting folds.

```
folds <- survML::generate_folds(n, V, sample_split)
cf_folds <- folds$cf_folds
table(cf_folds)
```



```
cf_folds
  1  2  3  4  5  6  7  8  9 10
50 50 50 50 50 50 50 50 50 50
```

```
ss_folds <- folds$ss_folds
table(ss_folds)
```

```
ss_folds
  0    1
250 250
```

```
V0_stackG_preds_scen3 <- CV_generate_full_predictions_landmark(
  time = time,
  event = event,
  X = X,
  landmark_times = landmark_times,
  approx_times = approx_times,
  nuisance = "stackG",
  cf_folds = cf_folds)
str(V0_stackG_preds_scen3)
```

List of 6

```
$ CV_full_preds_train:List of 10
..$ : num [1:450, 1:2] 0.977 0.922 0.983 0.986 0.981 ...
..$ : num [1:450, 1:2] 0.984 0.932 0.986 0.989 0.993 ...
..$ : num [1:450, 1:2] 0.983 0.93 0.989 0.985 0.991 ...
..$ : num [1:450, 1:2] 0.977 0.982 0.982 0.989 0.926 ...
..$ : num [1:450, 1:2] 0.936 0.985 0.988 0.985 0.991 ...
..$ : num [1:450, 1:2] 0.981 0.932 0.985 0.99 0.987 ...
..$ : num [1:450, 1:2] 0.973 0.928 0.975 0.978 0.968 ...
..$ : num [1:450, 1:2] 0.978 0.93 0.98 0.984 0.977 ...
..$ : num [1:450, 1:2] 0.983 0.941 0.981 0.987 0.981 ...
..$ : num [1:450, 1:2] 0.974 0.929 0.978 0.981 0.978 ...
$ CV_full_preds      :List of 10
..$ : num [1:50, 1:2] 0.977 0.986 0.986 0.986 0.986 ...
..$ : num [1:50, 1:2] 0.985 0.989 0.989 0.914 0.989 ...
..$ : num [1:50, 1:2] 0.987 0.989 0.985 0.991 0.985 ...
..$ : num [1:50, 1:2] 0.935 0.986 0.989 0.977 0.986 ...
..$ : num [1:50, 1:2] 0.981 0.982 0.988 0.991 0.988 ...
..$ : num [1:50, 1:2] 0.932 0.99 0.99 0.99 0.99 ...
..$ : num [1:50, 1:2] 0.978 0.975 0.975 0.978 0.975 ...
```

```

..$ : num [1:50, 1:2] 0.984 0.984 0.984 0.984 0.984 ...
..$ : num [1:50, 1:2] 0.927 0.941 0.987 0.987 0.941 ...
..$ : num [1:50, 1:2] 0.981 0.929 0.981 0.966 0.981 ...
$ CV_S_preds          :List of 10
..$ : num [1:50, 1:317] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:50, 1:317] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:50, 1:317] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:50, 1:317] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:50, 1:317] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:50, 1:317] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:50, 1:317] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:50, 1:317] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:50, 1:317] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:50, 1:317] 1 1 1 1 1 1 1 1 1 1 ...
$ CV_S_preds_train    :List of 10
..$ : num [1:450, 1:317] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:450, 1:317] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:450, 1:317] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:450, 1:317] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:450, 1:317] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:450, 1:317] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:450, 1:317] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:450, 1:317] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:450, 1:317] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:450, 1:317] 1 1 1 1 1 1 1 1 1 1 ...
$ CV_G_preds          :List of 10
..$ : num [1:50, 1:317] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:50, 1:317] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:50, 1:317] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:50, 1:317] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:50, 1:317] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:50, 1:317] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:50, 1:317] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:50, 1:317] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:50, 1:317] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:50, 1:317] 1 1 1 1 1 1 1 1 1 1 ...
$ CV_G_preds_train    :List of 10
..$ : num [1:450, 1:317] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:450, 1:317] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:450, 1:317] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:450, 1:317] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:450, 1:317] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:450, 1:317] 1 1 1 1 1 1 1 1 1 1 ...

```

```

..$ : num [1:450, 1:317] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:450, 1:317] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:450, 1:317] 1 1 1 1 1 1 1 1 1 1 ...
..$ : num [1:450, 1:317] 1 1 1 1 1 1 1 1 1 1 ...

```

```

CV_full_preds <- V0_stackG_preds_scen3$CV_full_preds
CV_full_preds_train <- V0_stackG_preds_scen3$CV_full_preds_train
CV_S_preds <- V0_stackG_preds_scen3$CV_S_preds
CV_G_preds <- V0_stackG_preds_scen3$CV_G_preds

indx <- 1:ncol(X)
vim_stackG_scen3 <- purrr::map_dfr(indxs, function(indx_i) {

  char_indx <- as.character(indx_i)
  indx <- as.numeric(strsplit(char_indx, split = ",")[[1]])

  CV_reduced_preds <- CV_generate_reduced_predictions_landmark(
    time = time,
    event = event,
    X = X,
    landmark_times = landmark_times,
    cf_folds = cf_folds,
    indx = indx,
    full_preds_train = CV_full_preds_train
  )

  purrr::map_dfr(vims, function(vim) {

    output <- switch(
      vim,
      brier = survML::vim_brier(
        time = time,
        event = event,
        approx_times = approx_times,
        landmark_times = landmark_times,
        f_hat = CV_full_preds,
        fs_hat = CV_reduced_preds,
        S_hat = CV_S_preds,
        G_hat = CV_G_preds,
        cf_folds = cf_folds,
        sample_split = sample_split,
        ss_folds = ss_folds

```

```

),
AUC = survML::vim_AUC(
  time = time,
  event = event,
  approx_times = approx_times,
  landmark_times = landmark_times,
  f_hat = purrr::map(CV_full_preds, ~ 1 - .x),
  fs_hat = purrr::map(CV_reduced_preds, ~ 1 - .x),
  S_hat = CV_S_preds,
  G_hat = CV_G_preds,
  cf_folds = cf_folds,
  sample_split = sample_split,
  ss_folds = ss_folds
)
)

output %>%
  dplyr::mutate(
    vim = vim,
    indx = char_indx
  )
})
})

kable(vim_stackG_scen3)

```

	landmark_time	var_est	cil	ciu	cil_1sided	p	large_predictions	small_predictions	time_index
25%...	11.647433	- 48.0233216	- 0.8357859	- 0.5211402	0.5157423	0.6389781	AUC 1		
		0.0232359	0.8822577	0.7441497					
75%...	28.959415	- 2.2964900	- 0.1483116	- 0.6600235	0.299232	0.6694613	AUC 1		
		0.0395380	0.2273877	0.1971864					
25%...	31.647433	- 0.3702634	- 0.0609005	- 0.6470971	-	-	brier 1		
		0.0145277	0.0899558	0.0778290	0.7655519	0.7510242			
75%...	48.959415	- 0.2525744	- 0.0544366	- 0.5976706	-	-	brier 1		
		0.0078612	0.0701590	0.0601432	0.4588184	0.4509572			
25%...	51.647433	- 48.4038432	- 0.8256020	- 0.5333407	0.5157423	0.6525586	AUC 2		
		0.0368164	0.8992348	0.7605807					
75%...	68.959415	- 2.2921101	- 0.1494274	- 0.6551906	0.299232	0.6681662	AUC 2		
		0.0382430	0.2259134	0.1957410					
25%...	71.647433	- 0.3702338	- 0.0611078	- 0.6450704	-	-	brier 2		
		0.0143174	0.0897425	0.0776162	0.7655519	0.7512345			

	landmark_time	var_est	cil	ciu	cil_1sided	p	large_prediction	small_prediction	metric
75%...	88.959415	-	0.2525823	-	0.0544111	-	0.5979917	-	brier 2
		0.0078877	0.0701865		0.0601705		0.4588184	0.4509307	
25%...	91.647433	-	48.6089949	-	0.8349161	-	0.5265145	0.6450702	AUC 3
		0.0293279	0.8935720		0.7546244				
75%...	18.959415	-	2.2819156	-	0.1426593	-	0.6796628	0.6745166	AUC 3
		0.0445933	0.2318459		0.2017407				
25%...	11.647433	-	0.3701883	-	0.0600785	-	0.6549412	-	brier 3
		0.0153420	0.0907625		0.0786369		0.7655519	0.7502099	
75%...	18.959415	-	0.2525934	-	0.0545965	-	0.5957487	-	brier 3
		0.0077037	0.0700038		0.0599876		0.4588184	0.4511147	
25%...	13.647433	-	48.4744128	-	0.8276873	-	0.5320001	0.6511018	AUC 4
		0.0353596	0.8984064		0.7596513				
75%...	18.959415	-	2.2886489	-	0.1530775	-	0.6406008	0.6643744	AUC 4
		0.0344512	0.2219798		0.1918302				
25%...	15.647433	-	0.3702504	-	0.0609146	-	0.6469504	-	brier 4
		0.0145122	0.0899391		0.0778124		0.7655519	0.7510397	
75%...	18.959415	-	0.2525850	-	0.0546247	-	0.5953935	-	brier 4
		0.0076744	0.0699735		0.0599575		0.4588184	0.4511440	
25%...	17.647433	-	48.9336093	-	0.8315080	-	0.5320823	0.6513592	AUC 5
		0.0356170	0.9027420		0.7633312				
75%...	18.959415	-	2.2779866	-	0.1551005	-	0.6312384	0.6619141	AUC 5
		0.0319908	0.2190822		0.1890028				
25%...	19.647433	-	0.3701904	-	0.0612045	-	0.6440994	-	brier 5
		0.0142162	0.0896369		0.0775113		0.7655519	0.7513357	
75%...	20.959415	-	0.2515475	-	0.0535139	-	0.6075421	-	brier 5
		0.0086571	0.0708282		0.0608327		0.4588184	0.4501613	
25%...	21.647433	0.0626643	53.6005103	-	0.9701978	-	0.4461739	0.5530777	AUC 6
			0.8448687		0.6989614				
75%...	28.959415	0.0675276	8407668	-	0.2764552	-	0.2632098	0.5623956	AUC 6
			0.1414000		0.1078100				
25%...	23.647433	-	0.3708026	-	0.0581462	-	0.6737027	-	brier 6
		0.0173368	0.0928199		0.0806842		0.7655519	0.7482151	
75%...	28.959415	-	0.2476899	-	0.0419051	-	0.7352083	-	brier 6
		0.0197873	0.0814798		0.0715613		0.4588184	0.4390311	
25%...	25.647433	-	48.2189872	-	0.8382487	-	0.5204491	0.6382635	AUC 7
		0.0225213	0.8832913		0.7449022				
75%...	28.959415	-	2.3978815	-	0.1397464	-	0.7030008	0.6821286	AUC 7
		0.0522053	0.2441570		0.2132963				
25%...	27.647433	-	0.3701592	-	0.0607811	-	0.6481660	-	brier 7
		0.0146364	0.0900540		0.0779288		0.7655519	0.7509155	

	landmark_time	var_est	cil	ciu	cil_1sided	p	large_predictions	small_predictions	time_index
75%...	28.959415	-	0.2522718	-	0.0546806	-	0.5942974	-	brier 7
		0.0075798		0.0698403		0.0598305	0.4588184	0.4512386	
25%...	29.647433	-	48.2753822	-	0.8358602	-	0.5230585	0.6411553	AUC 8
		0.0254131		0.8866863		0.7482163			
75%...	38.959415	-	2.5567116	-	0.1910015	-	0.5284003	0.6371288	AUC 8
		0.0072055		0.2054125		0.1735461			
25%...	31.647433	-	0.3701154	-	0.0609777	-	0.6462335	-	brier 8
		0.0144354		0.0898485		0.0777240	0.7655519	0.7511165	
75%...	38.959415	-	0.2526347	-	0.0544330	-	0.5977934	-	brier 8
		0.0078722		0.0701775		0.0601604	0.4588184	0.4509462	
25%...	33.647433	-	47.9949828	-	0.8309163	-	0.5253423	0.6435942	AUC 9
		0.0278520		0.8866203		0.7485530			
75%...	38.959415	-	2.2778692	-	0.1132045	-	0.7805305	0.7038053	AUC 9
		0.0738820		0.2609686		0.2308900			
25%...	35.647433	-	0.3701292	-	0.0606285	-	0.6496127	-	brier 9
		0.0147859		0.0902004		0.0780758	0.7655519	0.7507660	
75%...	38.959415	-	0.2526678	-	0.0554252	-	0.5857181	-	brier 9
		0.0068842		0.0691935		0.0591758	0.4588184	0.4519343	
25%...	37.647433	-	43.3371247	-	0.7865118	-	0.5282639	0.6452643	AUC 10
		0.0295220		0.8455558		0.7143592			
75%...	38.959415	-	2.2089314	-	0.1483075	-	0.6488404	0.6658495	AUC 10
		0.0359262		0.2201600		0.1905401			
25%...	39.647433	-	0.3702627	-	0.0609580	-	0.6465412	-	brier 10
		0.0144701		0.0898982		0.0777713	0.7655519	0.7510818	
75%...	48.959415	-	0.2528048	-	0.0548629	-	0.5927787	-	brier 10
		0.0074633		0.0697896		0.0597691	0.4588184	0.4513551	

4 Additional numerical experiments

4.1 Sensitivity to covariate correlations

4.1.1 Additive hazards

4.1.1.1 Global survival stacking

4.1.1.2 Aalen model